

UNIVERSITY OF SOUTHERN DENMARK

EMBODIED AI

Search and Rescue
Lego Mindstorm EV3

Nicolai Hoang Huy Nguyen
ningu20@student.sdu.dk

Adam Kolodziej
adkol23@student.sdu.dk

Group 18

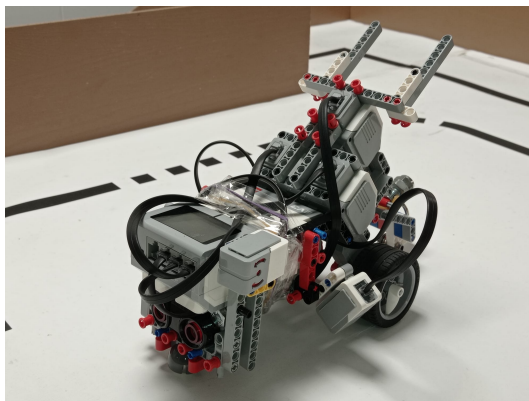
Handover Date: 21-12-2023

1 Introduction

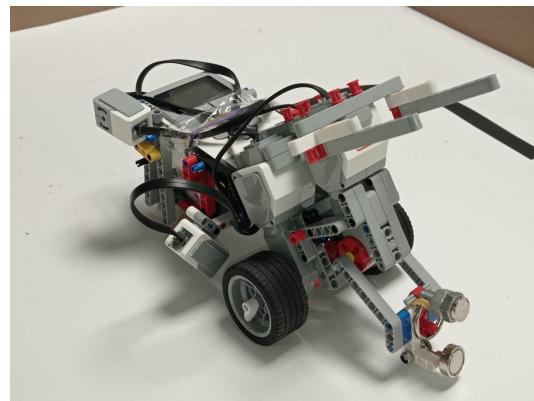
This project examines a search and rescue task for a self-designed mobile robot. The mobile robot must be built with the Lego Mindstorm EV3 kit provided. The robot's design should enable it to move around in a given environment. Additional sensor inputs from the kit should be employed to create a behavior-based architecture. The primary task for the robot is to navigate in a given map, following specific lines to reach the object that needs to be rescued. The next part is grabbing an object and coming back to the start point.

2 Mechanical Design

In this project, a few different designs were build. In order to go safely up and down of the ramp, it was decided to have the center of mass as low as possible. To achieve this, a traditional design was initially employed, with motors mounted parallel to the EV3 Brick. Unfortunately, this approach proved ineffective for climbing, particularly when transitioning from the ramp to the floor. During this phase, the color sensors were consistently collided with the edge of the ramp. To address this problem, the robot was completely rebuild. The final version can be seen in figure 1.



(a) The front of the robot



(b) The rear of the robot

Figure 1: Final version

We decided to put EV3 Brick higher to create space for the edge. At the same time to be able to build stiff, compact construction drive motors were tilted and mounted

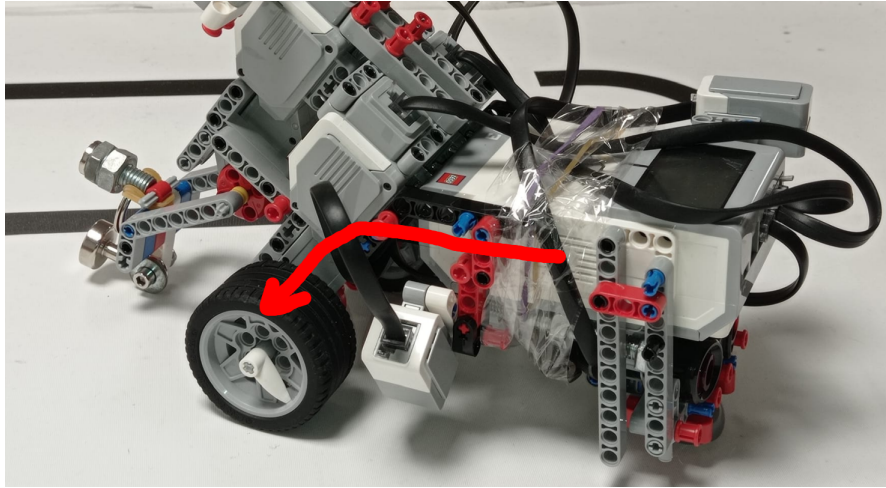


Figure 2: Forces

in the back. Obviously motors were mounted in the back but what is worth to underline - tilted motors also transferred weight of the EV3 in the direction which gave us even more friction 2. In the front we used metal ball, although it sometimes worked a bit roughly. We also had a version with the "blade" with the suspension - to be able to easier go on the ramp. This platform worked perfectly. It was capable of going up and down the hill.

2.1 Color sensors

Color sensors was one of the most challenging part of the whole build. Color sensors had to be mounted in the place which will allow to have constant reliable data and at the same time to be able to cross the edge of the ramp. We decided to place sensors as close as possible to the back driving wheels. It should have helped to have stable, more controllable movements while following the line because small robot turns result in small sensors turns. Having this stable platform we tried to put sensors in traditional way - 90 degrees to the surface the robot is driving. Unfortunately this type of mounting resulted in quite wide space between sensors. It was the reason why robot wasn't going straight but was moving around the line - there was a space without data from sensors. To get rid of this problem we wanted to mount them higher to have larger beam of light but this wasn't the solution since sensors theoretically should be about 1 cm above the surface (in our case they were

higher but still working fine). Unfortunately it wasn't possible in our construction. We decided to mount them tilted 45 degrees to the center. It gave us more surface covered with beam. This way we got the best results. After PID tuning the robot was able to drive reasonably stable. Due to problems with detecting the line's end we had to adjust the distance between sensors. This is explained in point 3. Generally speaking sensors were really unreliable. We even replaced devices a few times when we were tired of getting constantly changing data. As an example we can use the video attached to this report. As soon end of the line is reached, despite of both sensors looking at a white surface, the robot started turning which (we checked it by printing data in the terminal) meant that reading were different.

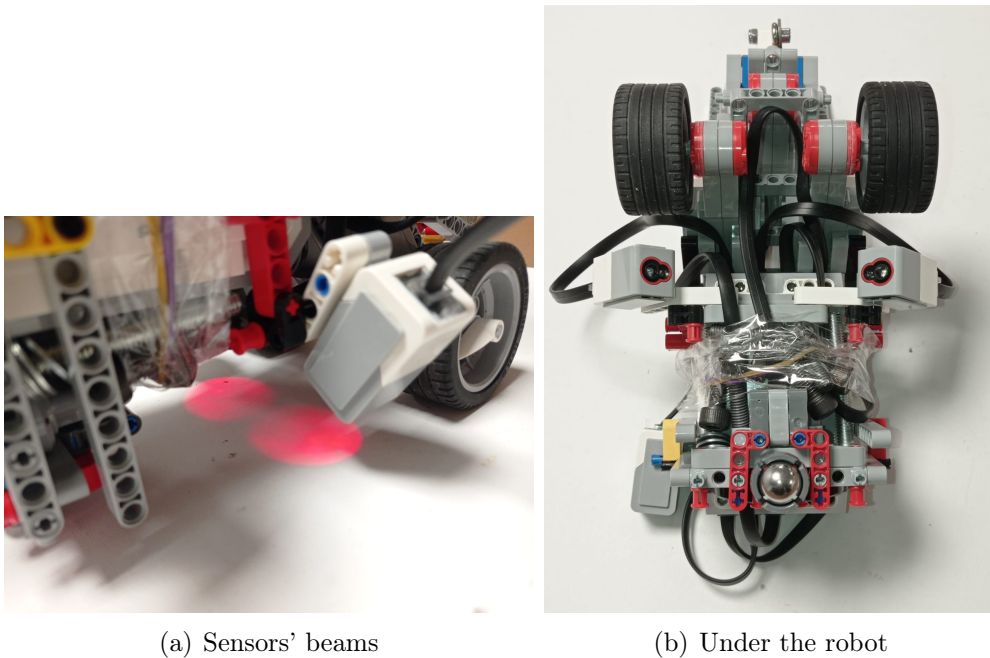


Figure 3: Color sensors

2.2 Ultrasonic sensor

To be able to find a can we used an ultrasonic sensor. It was mounted on the front of the robot relatively close to the ground. This position allows to see the can which is quite short. Moreover this placement wasn't influenced by the floor (the sensor was high enough).

Our first idea was to perform the scan just after reaching first floor. Since first floor

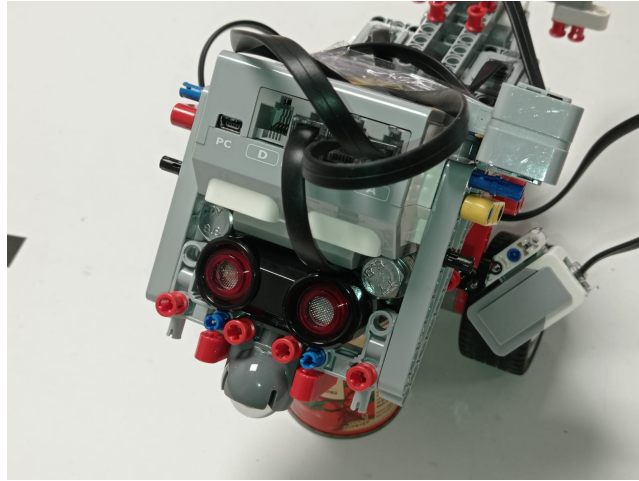


Figure 4: Ultrasonic sensor

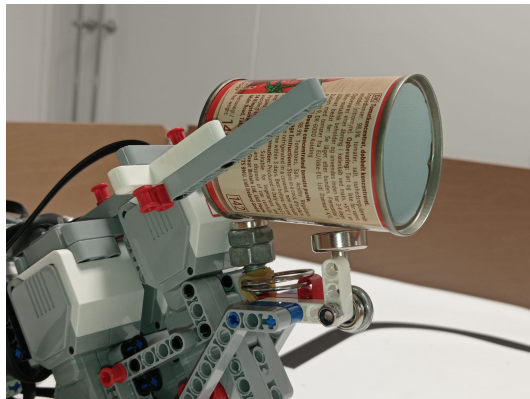
has walls, we were looking for sudden change of measured distance. We performed a few tests to determine if it's possible. Unfortunately, it turned out that sensor is too inaccurate on the distance close to length of the whole first floor map. We tried to improve data performing scan from left to right and from right to left (then theoretically it would be able to detect can from both sides and then determine it's center point). We even implemented it in the code but because one of the "eyes" is transmitter and the other one the receiver, it was hard to get reliable data (when you scan from left to right can was detected earlier than scanning the other way). Measured distance was also too noisy to trust it every time. It had to be simplified, which is explained in the point 3.

2.3 Gyro sensor

From the research we have done, many people have problems with these sensors - just look at the forums or YouTube. In case of this sensor it wasn't that important to mount it in specific way. Everything we needed to do was to keep correct orientation - to measure correct angle. Programming part is more important. All you can do with the sensor physically is to boot the robot only when it's on the flat surface. You can also reset angles by unplugging the sensor, waiting a few seconds and plugging it in again. However we had a feeling that readings are sometimes completely random.

2.4 Gripper

Our first attempt utilized typical LEGO gripper build with Lego Mindstorms EV3 Medium Servo Motor 45503 and worm gear. Unfortunately construction of our robot limited our imagination - it was tricky to mount that kind of gripper. Our final solution was more original than typical gripper. We have really strong neodymium magnets. We used third L-servo motor to move the gripper equipped with two magnets. These two magnets were strong enough to grab full can but not enough to hold it going down the ramp - too shaky. What is useful, can was attracted to the magnets from even 2 cm. Our main problem with this gripper was its weight. The robot struggled to go up the ramp having this gripper mounted. To get rid of this issue we added some counterweight in front of the robot (some screws).



(a) Can in the socket



(b) Grabbed can

Figure 5: The gripper

3 Control Architecture

The mobile robot is equipped with various components, as shown in figure 1. The EV3 Brick, the motors, and sensors form the foundation for the robot's sensory perception and actuation capabilities.

3.1 Sensor Integration

All sensor inputs are continuously read, and the values from the sensors are a crucial aspect of the system for it to function as a behavior-based architecture. The input sensors include two color sensors, an ultrasonic sensor, and a gyro sensor.

3.2 Behavioral Architecture

The system implements a behavior-based architecture, ideally consisting of various behavioral functions. A diagram of the behavior-based architecture can be seen in figure 6. Each of these behavior functions are designed to perform a specific task - most of them are running parallel. All the input sensors play a significant role for the robot.

Our PID controllers as an input use the difference between data from both color sensors. It was decided to implement two PID controllers for the line follower behavior. Throughout tests and experiences, it was found that the overall movement of our robot performed best with a PD controller ($K_p=37$, $K_d=3.3$). The second controller, also PD ($K_p=34$, $K_d=3$), was used during going up the ramps - it was supposed to be slower than typical one. Final tune for both controllers was achieved with manual tuning. Methods like Ziegler-Nichols (we tried it, it worked but not good enough) weren't successful. We were simply raising P values with turned off I and D so we can make robot as fast as possible with quite stable movement. Next to make it even more stable and resistant to "strange" parts of line like dots etc., we tried to add D and at the same time adjust P too. D should have slowed down movements and make it more smooth. From our experience from drones, too high D can cause fast oscillations and it happened during the tuning process several times.

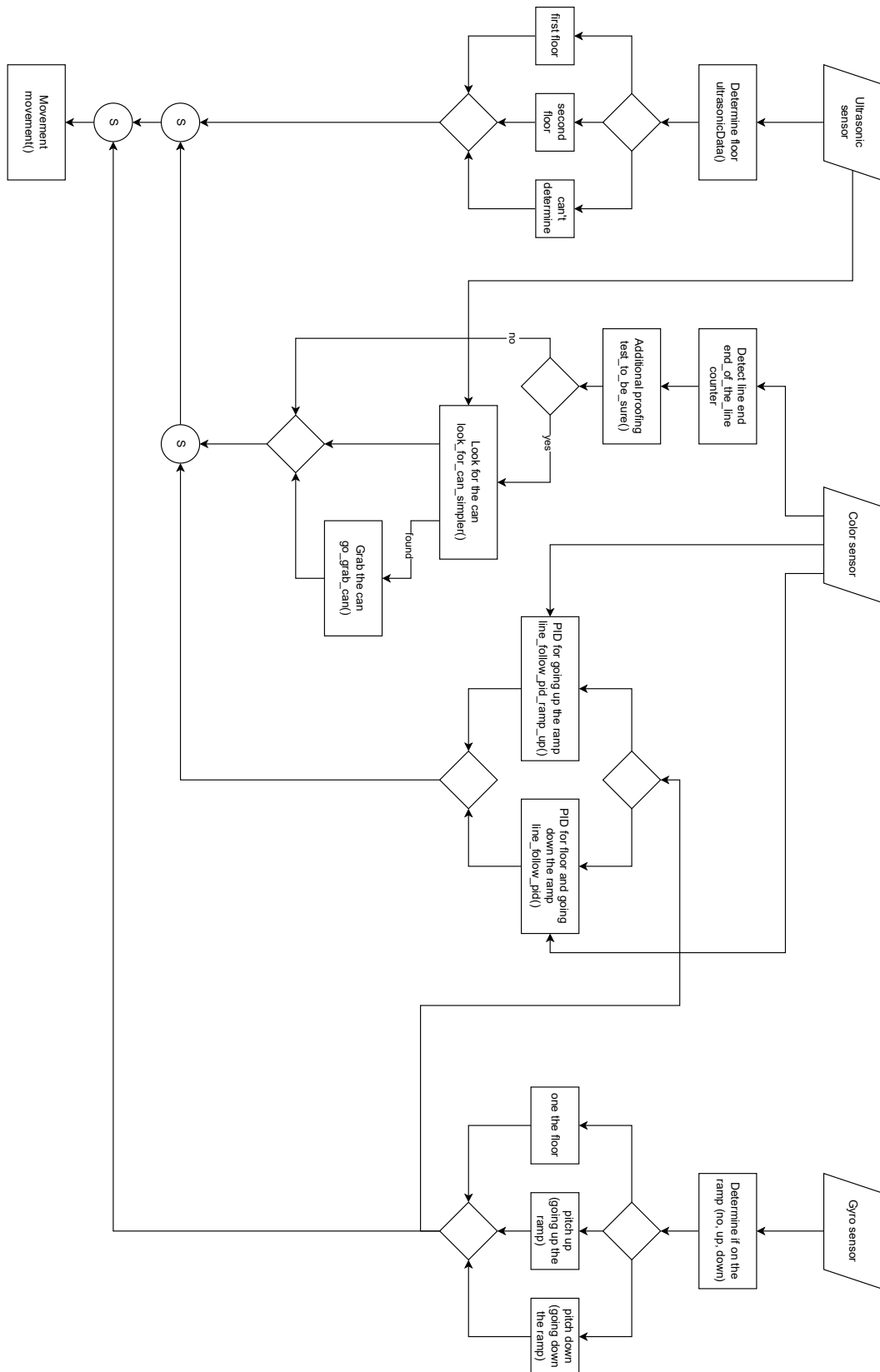


Figure 6: Behavioral architecture

In our case, it was hard to get well-working controller values for ramps and flat surface so we ended up using two controllers for each scenario. We also didn't use I because it only caused issues not helping at all. However, if we could use better sensors, we would definitely use I as well.

The color sensors were also used to check if the robot reached the end of the line. When checking for the end of the line, a counter is added and incremented for additional delay. This was done because some lines on the map were dotted. Therefore, it should be confirmed that the robot was at the end of a solid line and not on a dotted one. If the counter exceeds a threshold, the robot will perform additional checks to verify its position on the line. This is achieved by turning from left to right and detecting the line. If the robot doesn't detect the line, it will move forward slightly and repeat the test. This process is performed to eliminate noise and false information. This can be seen in Video 1 in A at 0 minutes and 24 seconds. If the line is detected, the robot continues following it - if not, it starts looking for the can. If the robot is at the end of the line, it will locate the can with the help of the ultrasonic sensor. To locate the can, the robot looks for a change in the output of the ultrasonic sensor by slowly turning from left to right. This change must exceed a threshold, indicating that the can has been found. Once the can is located, the robot moves straight toward it (till it reaches distanced based on the ultrasonic sensor), executes a 180-degree turn, and grabs the can with the magnets. This entire procedure can be observed in Video 2 in Appendix A at 0 minutes and 34 seconds. Subsequently, the robot retraces its path to re-engage in the line-following behavior.

The ultrasonic sensor also aids in floor level detection, distinguishing between the ground/first and second floors based on its output against a predetermined threshold. Since the ground floor has no walls, the output of the ultrasonic sensor on the ground floor will be very large. The idea is to measure the maximum distance on the first floor to establish the threshold that informs the robot that it is on the first floor when the ultrasonic sensor reaches this threshold.

The gyro sensor regulates the robot's speed when traversing inclines and declines

on the ramp. Sensor accumulates degrees so there code was written to be able to determine if the robot is on flat surface, going up the ramp or going down the ramp. Speed adjustments occur when the gyro output exceeds 7 (for acceleration) or falls below -7 (for deceleration). Acceleration enhances rescue efficiency, while deceleration mitigates the risk of collision with the end-wall during the descent of the ramp, thereby reducing failure rates. Increasing the speed will reduce the rescue time, and reducing the speed while going down will decrease the failure percentage, preventing the robot from colliding with the wall at the end of the ramp.

Each sensor input influences the robot's movements, contributing to its adaptive and responsive behavior.

4 Results

With the final design of the robot, it completed the entire search and rescue task in approximately 5 minutes and 20 seconds, achieving a success rate of 10%. A demonstration of the search and rescue task can be seen in Appendix A.

One problem with using the ultrasonic sensor to determine the floor level is that when the output reaches the threshold value for floor detection, it informs the robot that it is on the first floor, even though this information is inaccurate. This occurs when the robot is near the ramp or the poles on the ground floor, resulting in a low output value for the ultrasonic sensor. Additionally, this means that the robot performs the 'detect line end' behavior, which ideally should be executed on the first floor first. This can be observed in Video 1 in Appendix A at 1 minute and 1 second.

The low success rate is attributed to the poor gyro sensor. When running on a flat surface, the gyro sensor tends to increment rapidly, triggering unintended movements and acceleration. Consequently, the robot deviates from the intended path. The PID-controller could also be tuned more effectively. For this project, several PID controllers were tuned, and the Ziegler–Nichols method for PID tuning was implemented but showed poor performance.

Additionally, the use of the ultrasonic sensor for can detection introduced minor

issues, primarily stemming from sensor-generated noise. To address this, a potential solution involves implementing a filter to mitigate the impact of the noise.

5 Conclusion

We discussed the design as well as the code in previous chapters so in this section we will only sum up the whole project.

This project investigated a search and rescue task for a self-designed mobile robot built with the Lego Mindstorms EV3 kit. After considering and testing various designs, the decision was made to stick with the one described in section 2. The integration of color-, ultrasonic- and gyro sensor formed the sensory backbone of the robot. The control architecture implemented was a behavior-based approach, where different sensors triggered specific behaviors. The robot achieved an overall success rate of 10%. The major issue leading to this low success rate was identified as the poor performance of the gyro sensor. Furthermore, extensive tuning of the PID controllers was conducted. Final version used two controllers with different tune - one for standard line following and the second one for ramps. For future projects with the Lego Mindstorms EV3 kit, it would be beneficial to investigate the gyro sensor and explore alternatives for its replacement. Additionally, fine-tuning the integration of sensors could enhance overall performance, potentially resulting in a higher success rate. Alternative idea is to use additional color sensor instead of gyro sensor. That would be followed by completely different behavioral concept but we would get two very important benefits. First - get rid of problematic gyro sensor. Second, more significant - we could use third color sensor to achieve more reliable end of the line detection. Then it would be possible to mount color sensors used for line following even more optimally.

Appendix A

Test drive

Video 1

Video 2